

GenAI: Environment Setup

This module explains how **Generative AI** assists in creating and configuring test environments efficiently.

Role of GenAI in Environment Setup

Test Environment Setup involves preparing:

- Hardware
- Software
- OS
- Databases
- Tools
- Test data

Generative AI helps by:

- Generating environment configuration files
 - Suggesting tool installations
 - Creating Dockerfiles & docker-compose
 - Writing setup scripts
 - Detecting missing dependencies
-

How GenAI Supports Environment Setup

Area	GenAI Contribution
Tool installation	Generates commands
Containerization	Creates Dockerfiles
CI/CD setup	Generates pipelines
Dependency management	Creates requirements files
Troubleshooting	Suggests fixes

Environment Setup: Example

Scenario: Python Automation Testing Environment

Prompt

Create environment setup for Python automation testing with pytest.

GenAI Output

```
pip install pytest
pip install selenium
pip install requests
```

Create requirements.txt

```
pytest
selenium
requests
```

Example: Dockerized Test Environment

```
FROM python:3.11
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN pip install -r requirements.txt
```

```
CMD ["pytest"]
```

Example: CI Pipeline Setup

```
name: Run Tests
```

```
on: [push]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

- uses: actions/checkout@v3
- name: Install deps
 - run: pip install -r requirements.txt
- name: Run Tests
 - run: pytest

Copilot can generate this automatically.

Benefits

- Faster setup
- Fewer errors
- Standardized environments

Limitations

- Needs review
- May not match infrastructure

Quick Check

Q1. What is test environment setup?

Preparing tools and systems for testing

Q2. True/False

GenAI can generate Dockerfiles.
True

Q3. Best practice?

Review and customize AI output

Summary

Generative AI accelerates environment setup by:
Generating configs
Creating scripts
Reducing errors
But **human validation is essential.**
